

Aula 09 - Design Responsivo

Descrição: Nesta aula iremos tratar do conceito mobile-first, breakpoints (576px, 768px, 1024px) e media queries. Vamos realizar uma prática de tornar 3 páginas responsivas para mobile, tablet e desktop.

1. Abertura da Aula

Hoje entraremos na etapa de **Frontend: HTML, CSS, Responsividade**. Pensem no cenário do nosso projeto final: vocês fazem parte da equipe de desenvolvimento de uma empresa júnior de tecnologia e apresentarão uma proposta de inovação digital. **Imaginem o investidor abrindo a aplicação web funcional de vocês no celular dele durante a apresentação.** Se o sistema estiver quebrado, com botões minúsculos e texto ilegível, a credibilidade do projeto vai a zero.

Hoje, vamos focar no **design de interface para interação de subsistemas** e aprender a reconhecer requisitos de **usabilidade e qualidade**. O objetivo desta aula é **garantir que a interface do nosso sistema se adapte perfeitamente a qualquer tamanho de tela**, seja um celular, um tablet ou um monitor ultrawide, utilizando a abordagem Mobile-First e Media Queries.

2. Conceitos Fundamentais

Conceito 1: Abordagem Mobile-First

No desenvolvimento web moderno, a abordagem **Mobile-First (Primeiro para Dispositivos Móveis)** é uma estratégia de design e programação onde começamos a criar a interface **focando primeiro nas telas menores** (celulares) e, progressivamente, adicionamos complexidade e novos layouts para telas maiores (tablets e desktops).

Essa prática **inverte a lógica antiga** de criar para o desktop e depois tentar "espremer" o conteúdo para o celular. Ao priorizar o mobile, garantimos que **o conteúdo essencial seja carregado mais rápido**, economizando dados do usuário, e forçamos o desenvolvedor a focar na usabilidade primordial do sistema, o que é um requisito de qualidade essencial no mercado atual. Desenvolver *Mobile-First* significa que o código CSS padrão (fora de qualquer regra condicional) será o código do celular.

Demonstração:

Vejam este exemplo simples de CSS. O fundo padrão é azul (celular), mas muda para verde em telas maiores.

```
CSS
/* CSS Padrão (Mobile-First) - Aplicado primeiro em todas as telas */
body {
  background-color: lightblue;
  font-size: 16px;
  padding: 10px;
}

/* Regra condicional para telas maiores que 768px (Tablets e Desktops) */
@media (min-width: 768px) {
  body {
    background-color: lightgreen;
    padding: 30px;
  }
}
```

Explicação do código: O navegador lê de cima para baixo. Primeiro, ele aplica o `lightblue`. Se a tela do usuário tiver 768 pixels de largura ou mais, a regra `@media` é ativada, substituindo a cor de fundo para `lightgreen`.

Exercício:

Abram o editor de código. Crie um arquivo HTML com um botão `<button>Enviar</button>`. No CSS, faça com que esse botão tenha 100% de largura na tela padrão (mobile). Em seguida, criem uma regra `@media (min-width: 768px)` para que o botão passe a ter apenas 200px de largura.

Depois, adicionem um parágrafo de texto abaixo do botão. No mobile, o texto deve estar alinhado à esquerda. A partir de 1024px (desktop), o texto deve ficar centralizado e a fonte deve aumentar para 20px.

Conceito 2: Breakpoints (Pontos de Quebra)

Os **Breakpoints** são os **valores exatos de largura de tela** onde o layout do nosso site "quebra" ou precisa ser reorganizado para continuar oferecendo uma boa experiência. No mercado de tecnologia, nós **não criamos regras para cada modelo específico de celular** (pois existem milhares). Em vez disso, agrupamos os dispositivos em categorias de tamanhos utilizando breakpoints padrão.

Os três breakpoints mais utilizados no mercado e em frameworks (como o Bootstrap) são: **576px** (limite para celulares em modo paisagem), **768px** (tablets em modo retrato) e **1024px** (laptops e desktops). Compreender e aplicar esses pontos garante que nossa aplicação atenderá aos requisitos não funcionais de interface e usabilidade em múltiplas plataformas.

Demonstração:

Observem como estruturamos nosso CSS para abraçar essas três resoluções.

CSS

```
/* 1. Mobile-First (Celulares em pé - até 575px) */
.container { width: 100%; display: block; }

/* 2. Dispositivos pequenos / Celulares deitados (Breakpoint: 576px) */
@media (min-width: 576px) {
  .container { max-width: 540px; }
}

/* 3. Tablets (Breakpoint: 768px) */
@media (min-width: 768px) {
  .container { max-width: 720px; display: flex; }
}

/* 4. Desktops (Breakpoint: 1024px) */
@media (min-width: 1024px) {
  .container { max-width: 960px; }
}
```

Explicação do código: Começamos com uma largura fluida (100%). Conforme a tela cresce, os breakpoints (576, 768, 1024) vão assumindo o controle, limitando a largura máxima para que o conteúdo não fique esticado demais em telas grandes, além de alterar o comportamento de exibição (de `block` para `flex`).

Exercício:

Crie três caixas (divs) com a classe `.card`. No mobile, elas devem ficar empilhadas uma em cima da outra. Usem o breakpoint de `768px` para fazer com que elas fiquem lado a lado (usando `display: flex`).

Agora adicionem o breakpoint de `1024px`. Quando a tela atingir esse tamanho desktop, as caixas devem ganhar uma margem maior entre elas e uma borda mais grossa. Testem redimensionando a janela do navegador.

Conceito 3: Media Queries

São uma das ferramentas mais importantes do CSS moderno. De forma simples, elas funcionam como **filtros condicionais** que dizem ao navegador: **"Aplique estes estilos apenas se a tela do usuário tiver determinadas características"** (como uma largura ou altura específica).

É o conceito central do **Design Responsivo**, permitindo que um único site se adapte perfeitamente a um iPhone, um tablet ou um monitor UltraWide de 30 polegadas.

Como elas funcionam (A Sintaxe)

Uma media query geralmente é composta por um **tipo de mídia** e uma ou mais **expressões** que verificam as condições dos dispositivos.

@media (condição) { ...estilos aqui... }

Os componentes principais são:

1. **@media:** A regra que inicia a consulta.
2. **Media Type:** Define o dispositivo (ex: `screen` para telas, `print` para impressoras). Se não for especificado, o padrão é `all`.
3. **Media Features (Condições):** As características que você quer testar, como:
 - `min-width`: "Se a tela tiver no mínimo X de largura".
 - `max-width`: "Se a tela tiver no máximo X de largura".
 - `orientation`: Verifica se o celular está em pé (*portrait*) ou deitado (*landscape*).

Demonstração:

Mobile-First (O padrão atual)

Nesta abordagem, você escreve o CSS para celular primeiro e usa a media query para "adicionar" estilo conforme a tela cresce:

CSS

```
/* Estilo para todos (foco no celular) */
```

```
body { background-color: white; }
```

```
/* Quando a tela for maior que 768px (Tablets/Laptops) */
```

```
@media (min-width: 768px) {
```

```
  body { background-color: lightblue; }
```

```
}
```

Mudando a Orientação

Você pode alterar o layout dependendo de como o usuário segura o celular:

CSS

```
@media (orientation: landscape) {
```

```
  .menu { display: flex; } /* Mostra o menu de lado se o celular estiver deitado */
```

```
}
```

Por que isso é importante?

Antigamente, as empresas criavam dois sites: `www.site.com` e `m.site.com` (específico para mobile). Com as Media Queries, você mantém **um único código HTML** e apenas ajusta a "roupa" (o CSS) conforme o tamanho da janela.

Isso melhora a manutenção do código, ajuda no SEO (Google prefere sites responsivos) e garante que nenhum usuário tenha uma experiência ruim, independente do aparelho que use.

3. Demonstração Aplicada

Vamos ver isso funcionando na prática usando uma ferramenta de desenvolvimento profissional do Google Chrome.

1. Cliquem com o botão direito na página e selecionem "**Inspecionar**" (ou pressionem `F12`).
 2. Procurem o ícone de "**Dispositivos**" (um celular desenhado ao lado de um tablet) no canto superior esquerdo do DevTools, ou usem `Ctrl+Shift+M`.
 3. Isso ativa o modo de design responsivo. No topo da tela, vocês verão uma barra onde podem escolher modelos específicos (como iPhone 12 ou iPad Air) ou digitar os nossos **breakpoints** manuais: 576, 768 e 1024.
 4. **Vamos redimensionar a tela!** Notem como, ao arrastar a barra e passar de 767px para 768px, a estrutura muda instantaneamente de empilhada para lado a lado. É assim que garantimos a qualidade do código visualmente antes de entregar ao cliente.
-

4. Casos Reais do Mercado

- **E-commerce (Lojas Virtuais):** Grandes plataformas de vendas, como a **Amazon** ou o **Mercado Livre**, usam **Mobile-First** agressivamente. Se você acessar pelo celular, **a barra de pesquisa domina o topo** e os menus complexos são escondidos em um botão "**hambúrguer**" (três linhas). Se a conversão (venda) cair 1% por causa de um botão difícil de clicar no celular, a empresa perde milhões. **A responsividade aqui é puramente dinheiro e sobrevivência.**
 - **Sistemas de Gestão (Dashboards SaaS):** Quando criamos um painel administrativo (**dashboard**), no desktop ele exibe tabelas com 10 colunas, gráficos gigantes e menus laterais. No **mobile**, uma tabela de 10 colunas quebraria o layout. O mercado resolve isso transformando as linhas da tabela em "**cards**" individuais no celular através de **Media Queries**, ou escondendo colunas menos importantes, garantindo a integridade e usabilidade da informação.
-

5. FAQ -- Dúvidas

1. **Qual a diferença entre usar `min-width` e `max-width` nas media queries?**
 - `min-width` aplica a regra de um tamanho específico *para cima* (ideal para Mobile-First).
Exemplo: `min-width: 768px` afeta tablets e desktops. `max-width` aplica a regra do tamanho *para baixo* (ideal para Desktop-First). Recomenda-se sempre padronizar em `min-width`.
2. **Eu preciso testar o site no meu celular físico toda vez?**
 - Não necessariamente. O DevTools do navegador é excelente e simula muito bem 95% dos casos. Porém, antes de apresentar para os investidores na feira, abra o sistema no celular físico da equipe para testar a sensação real do toque (touch).

3. Por que minhas media queries não estão funcionando?

- Provavelmente é a ordem no arquivo CSS. **O CSS é lido de cima para baixo (Cascata)**. As media queries devem ficar **sempre no final do seu arquivo CSS** para poderem sobrescrever o código padrão.

4. O que é aquela tag `<meta name="viewport">` no HTML?

- Sem ela, o celular finge que é um monitor de desktop e diminui tudo para caber na tela. A tag `viewport` avisa ao navegador móvel: "Trate a largura da tela como a largura do dispositivo e não aplique zoom". **É obrigatória para a responsividade funcionar.**

5. Posso usar porcentagens (%) em vez de media queries?

- **Eles trabalham juntos!** Porcentagens deixam as larguras fluidas (ex: ocupar 50% da tela), enquanto as *media queries* alteram a estrutura (ex: mudar de 50% para 100% quando a tela ficar muito pequena).

6. Desafio Prático

Contexto: Nossa aplicação deve conter funcionalidades no front-end e back-end. Para o front-end, vocês precisarão tornar responsivas três páginas essenciais do nosso sistema: a **Página de Login**, o **Dashboard Inicial** e o **Formulário de Cadastro de Produto/Usuário** (que compõe a estrutura CRUD que faremos no futuro).

Requisitos:

- Vocês devem aplicar os breakpoints ensinados: `576px`, `768px` e `1024px`.
- As 3 páginas devem ter um layout distinto para Mobile (tela pequena), Tablet (tela média) e Desktop (tela grande).
- **Login:** Formulário ocupando 100% da largura no mobile, e centralizado em uma caixa menor no desktop.
- **Dashboard:** Menus empilhados no mobile, organizados lado a lado no desktop.
- **Formulário:** Campos de input em uma coluna no mobile, duas colunas no tablet e desktop.

Resultado Esperado:

Ao abrir as 3 páginas no Google Chrome e utilizar o DevTools para inspecionar nos tamanhos de 375px (mobile), 800px (tablet) e 1200px (desktop), as páginas não devem conter barra de rolagem horizontal e os elementos devem estar harmoniosos e legíveis.

Sugestão de Solução:

Comecem sempre pelo CSS do mobile. Estruturem as `divs` corretamente no HTML. Só depois de o mobile estar perfeito, escrevam o `@media (min-width: 768px)` no fim do arquivo e façam os ajustes para o tablet.

7. Conexão com a Situação de Aprendizagem

ATENÇÃO: Este conteúdo está diretamente ligado à nossa entrega final.

Vocês precisam entregar um sistema web funcional acompanhado da documentação. Lembrem-se de que a aplicação será avaliada por potenciais investidores. O investidor acessará a aplicação, muito provavelmente, pelo smartphone. A **capacidade de criar um layout responsivo demonstra profissionalismo e o cumprimento dos requisitos de qualidade e usabilidade do software**. O front-end que construímos e adaptamos hoje será a "casca" que conectaremos posteriormente ao nosso banco de dados relacional e ao back-end em PHP.

8. Preparação para a Próxima Aula

Hoje **dominamos a parte visual e estática da nossa interface** (HTML, CSS e Responsividade). Na próxima aula, o foco será acelerar o desenvolvimento de interfaces utilizando **Bootstrap** (ou Tailwind), focando em produtividade e padronização através de componentes prontos e sistemas de layout inteligentes.